



BITS Pilani
Pilani Campus

Deep Neural Network

Dr. Monali Mavani

Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Content



- Neural architecture search
 - Search spaces
 - Search Strategy
 - Performance Estimation Strategy
- Time series analysis
 - Components of TSA
 - Time series decomposition
 - TSA using deep learning

Neural architecture search



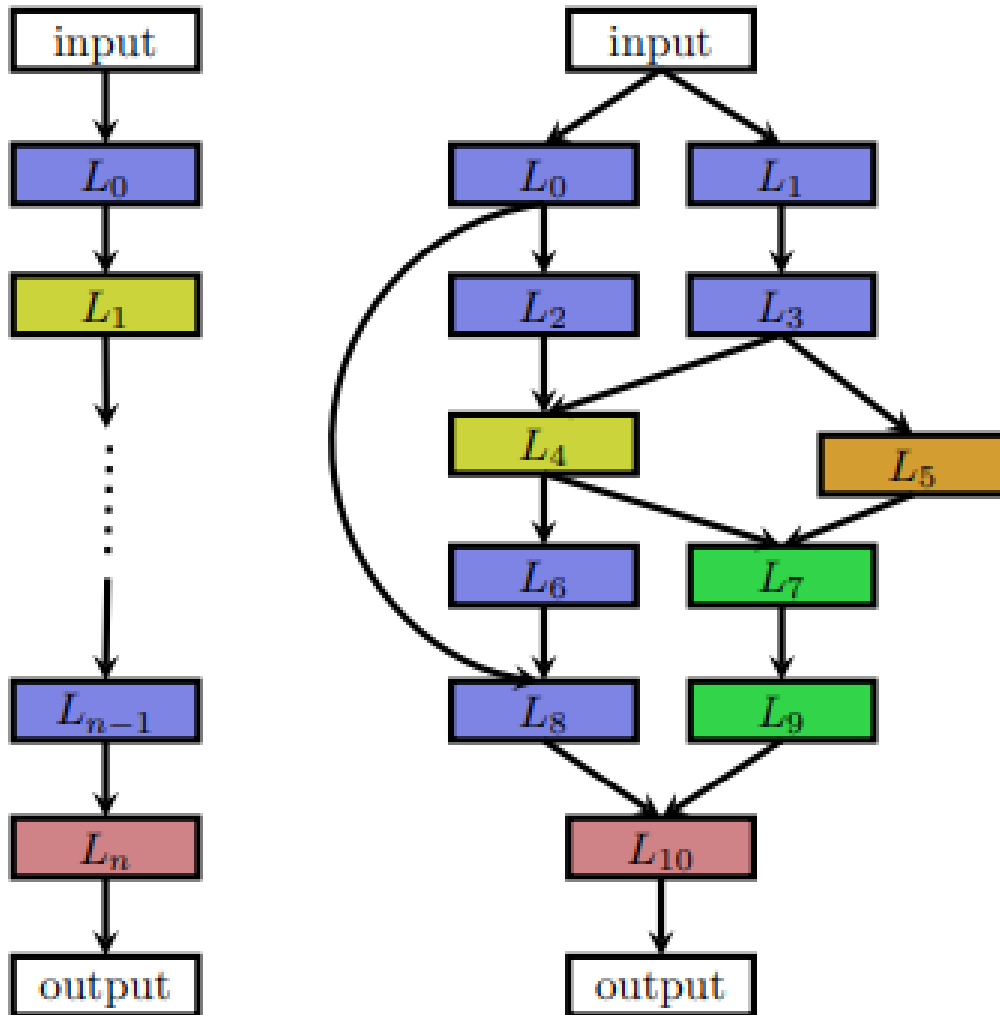
- Technique for automating the design of ANN
- NAS has been used to design networks that are on par or outperform hand-designed architectures
- Methods for NAS can be categorized as
 - *Search space* defines the type(s) of ANN that can be designed and optimized.
 - *Search strategy* : approach used to explore the search space.
 - *Performance estimation strategy* evaluates the performance of a possible ANN from its design (without constructing and training it).
- NAS is closely related to hyper parameter optimization and meta-learning and is a subfield of automated machine learning (AutoML)

Search spaces



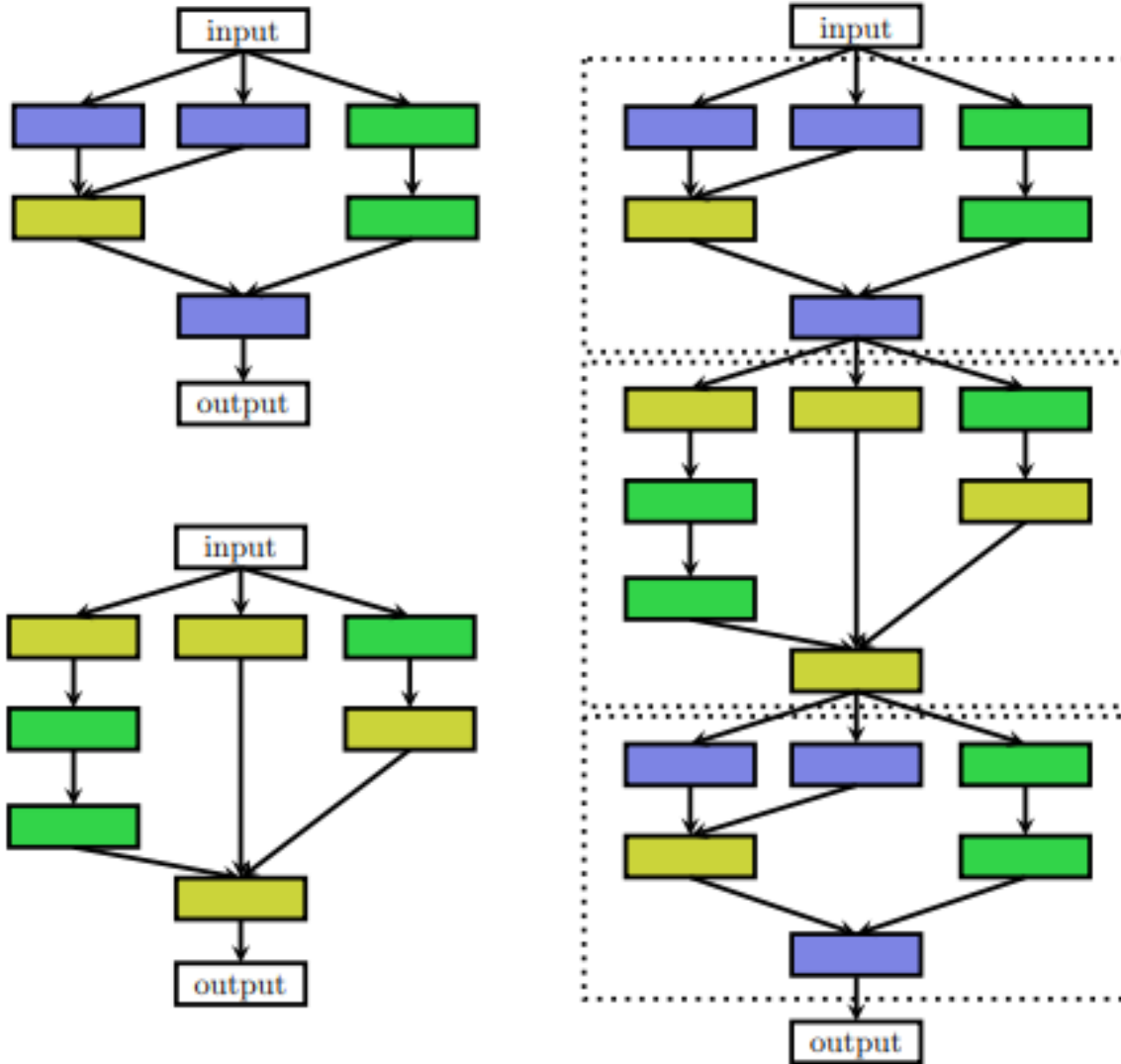
- Search space defines which neural architectures a NAS approach might discover in principle
- Search spaces can be broadly classified into two categories
 - The first category of search spaces is defined for the **graphs** that represent an entire neural architecture- **chained search space (global search space)**
 - An architecture is a **combination of few cells** which are repeated to build the complete network. –cell search space
- In principle the global search space encompasses the cell-based search space

Chained search space



- Each node in the graphs corresponds to a layer in a neural network, e.g., a convolutional or pooling layer
- Different layer types are visualized by different colors
- An edge from layer L_i to layer L_j denotes that L_j receives the output of L_i as input
- Left: an element of a chain-structured space
- Right: an element of a more complex search space with additional layer types and multiple branches and skip connections.

Cell search space



- Left: Two different cells, e.g., a normal cell (top) and a reduction cell (bottom)
- Normal cell preserves the dimensionality of the input
- Reduction cell reduces the spatial dimension
- Final architecture is then built by stacking these cells in a predefined manner
- Right: an architecture built by stacking the cells sequentially
- Design problem
 - How many cells shall be used
 - How should they be connected to build the actual model

Search space



- It is then parametrized by
 1. maximum number of layers n
 2. type of operation every layer executes, e.g., pooling, convolution, or more advanced operations like dilated convolutions
 3. hyperparameters associated with the operation, e.g., number of filters, kernel size and strides for a convolutional layer or simply number of units for fully-connected networks
- Note that the parameters from (iii) are conditioned on (ii), hence the parametrization of the search space is not fixed-length but rather a conditional space.

Search Strategy



- Random search
- Bayesian optimization
- Evolutionary method
- Reinforcement learning (RL)
- Gradient-based methods

Performance Estimation Strategy



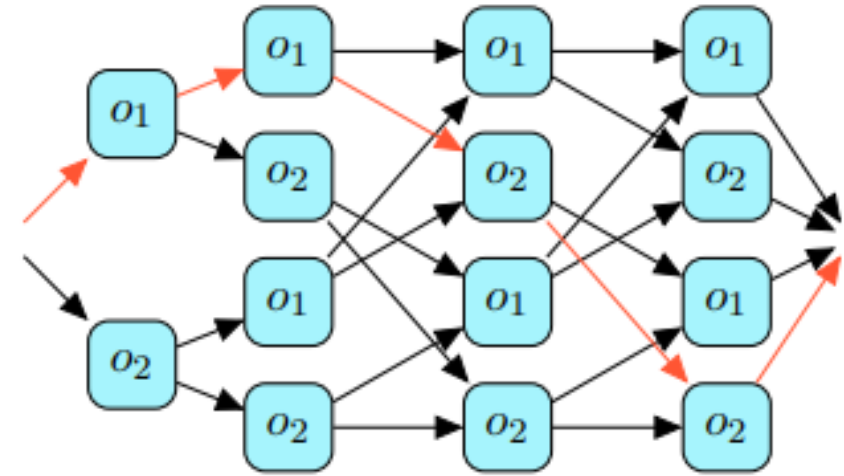
- The simplest way is to train A on training data and evaluate its performance on validation data.
- However, training each architecture to be evaluated from scratch frequently yields computational demands in the order of thousands of GPU days for NAS
- This naturally leads to developing methods for speeding up performance estimation

Speed-up method	How are speed-ups achieved?
Lower fidelity estimates	Training time reduced by training for fewer epochs, on subset of data, downscaled models, downscaled data, ...
Learning Curve Extrapolation	Training time reduced as performance can be extrapolated after just a few epochs of training.
Weight Inheritance/ Network Morphisms	Instead of training models from scratch, they are warm-started by inheriting weights of, e.g., a parent model.
One-Shot Models/ Weight Sharing	Only the one-shot model needs to be trained; its weights are then shared across different architectures that are just subgraphs of the one-shot model.

One shot strategy



- It trains a single neural network during the search process
- This neural network is then used to derive architectures throughout the search space as candidate solutions to the optimization problem
- During the network parameter update step, a batch is selected and an architecture is sampled from the search space.
- Only the parameters of this architecture are updated, the remaining parameters remain unchanged

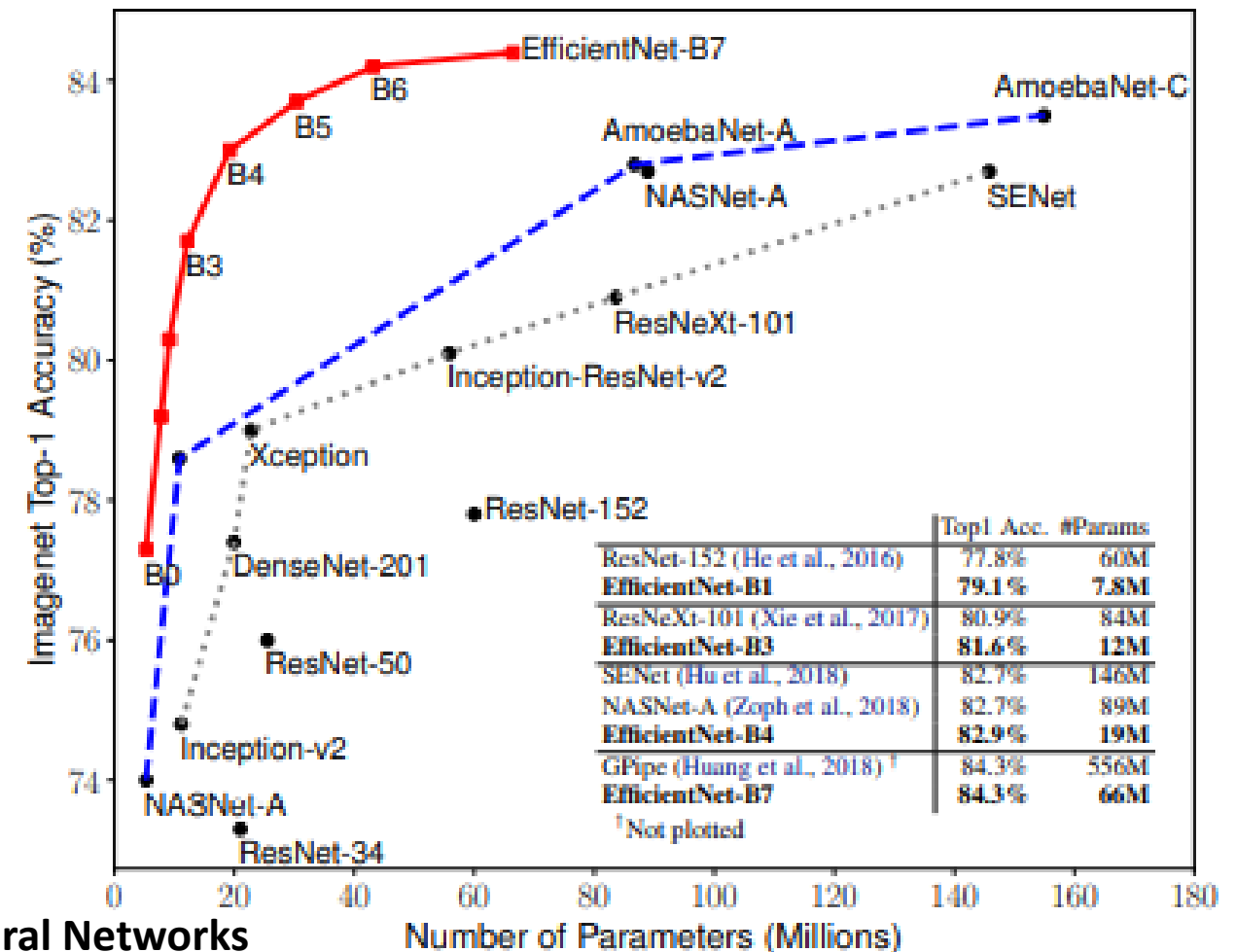


- Example for weight sharing with only two operations in the chain-structured search space
- Every box has its own weights
- E.g. the red one: $o1 \rightarrow o1 \rightarrow o2 \rightarrow o2$ is one architecture in the search space.
- Thus, weights are shared across different architectures

NAS example: EfficientNets



- It uses neural architecture search to design a new baseline network and scale it up to obtain a family of models, called EfficientNets
- Achieved much better accuracy and efficiency than previous ConvNets.
- EfficientNet-B7 achieves state-of-the-art 84.3% top-1 accuracy on ImageNet while being 8.4x smaller and 6.1x faster on inference than the best existing ConvNet.
- EfficientNets also transfer well and achieve state-of-the-art accuracy on CIFAR-100 (91.7%), Flowers (98.8%), and 3 other transfer learning datasets, with an order of magnitude fewer parameters.



EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

Time series analysis



- **Time series** is a series of data points indexed (or listed or graphed) in time order
- **Time series analysis is a** statistical technique that uses time-series data for explaining the past or forecasting future events.
- The prediction is a function of time (days, months, years, etc.)
- No *causal* variable; examine past behavior of a variable and attempt to predict future behavior

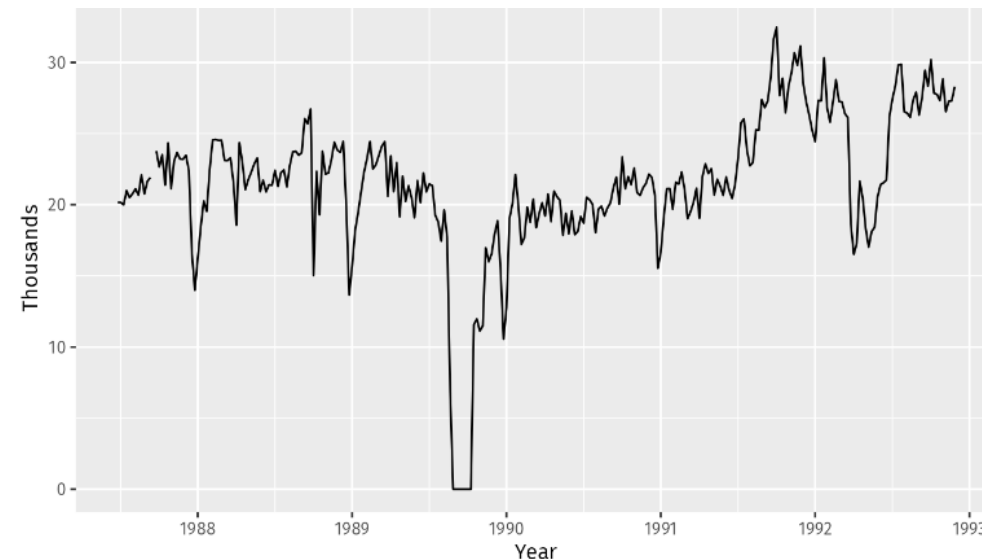


Figure 2.1: Weekly economy passenger load on Ansett Airlines.

Components of TSA

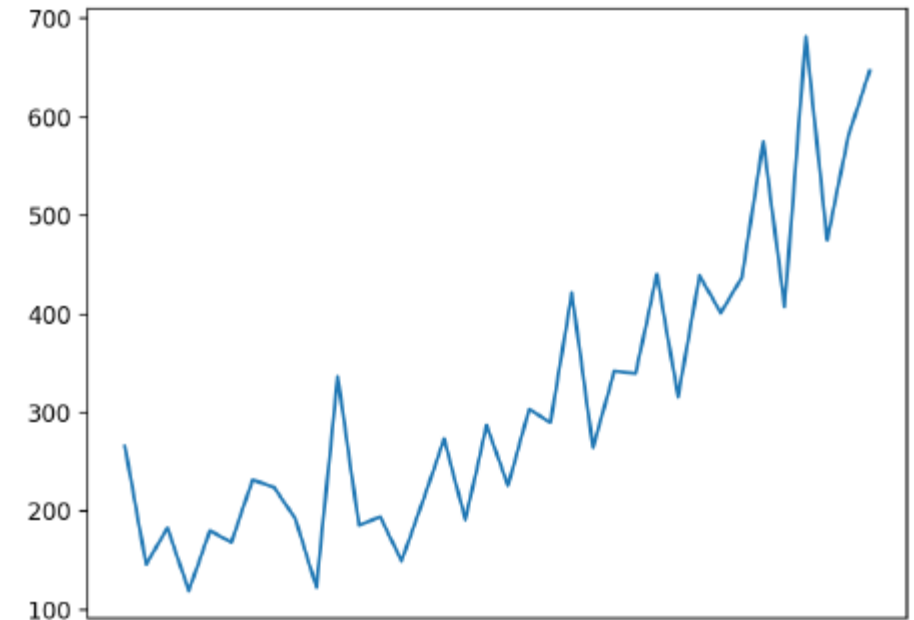


Time Frame (How far can we predict?)

- short-term (1 - 2 periods), medium-term (5 - 10 periods), long-term (12+ periods),
No line of demarcation

Trend

- Gradual, long-term movement (up or down) of demand
- Easiest to detect
- Does not have to be linear
- Line Plot of the Monthly Shampoo Sales Dataset

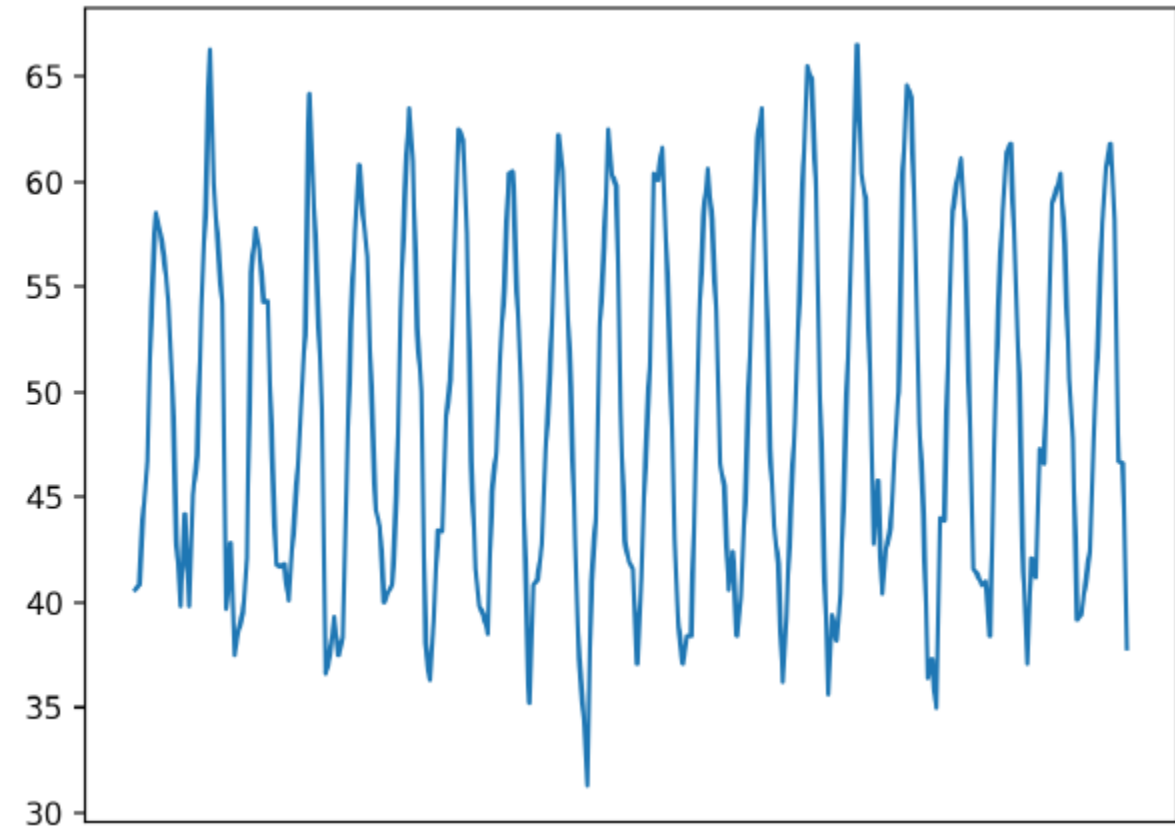


Components of TSA



Seasonal Variation

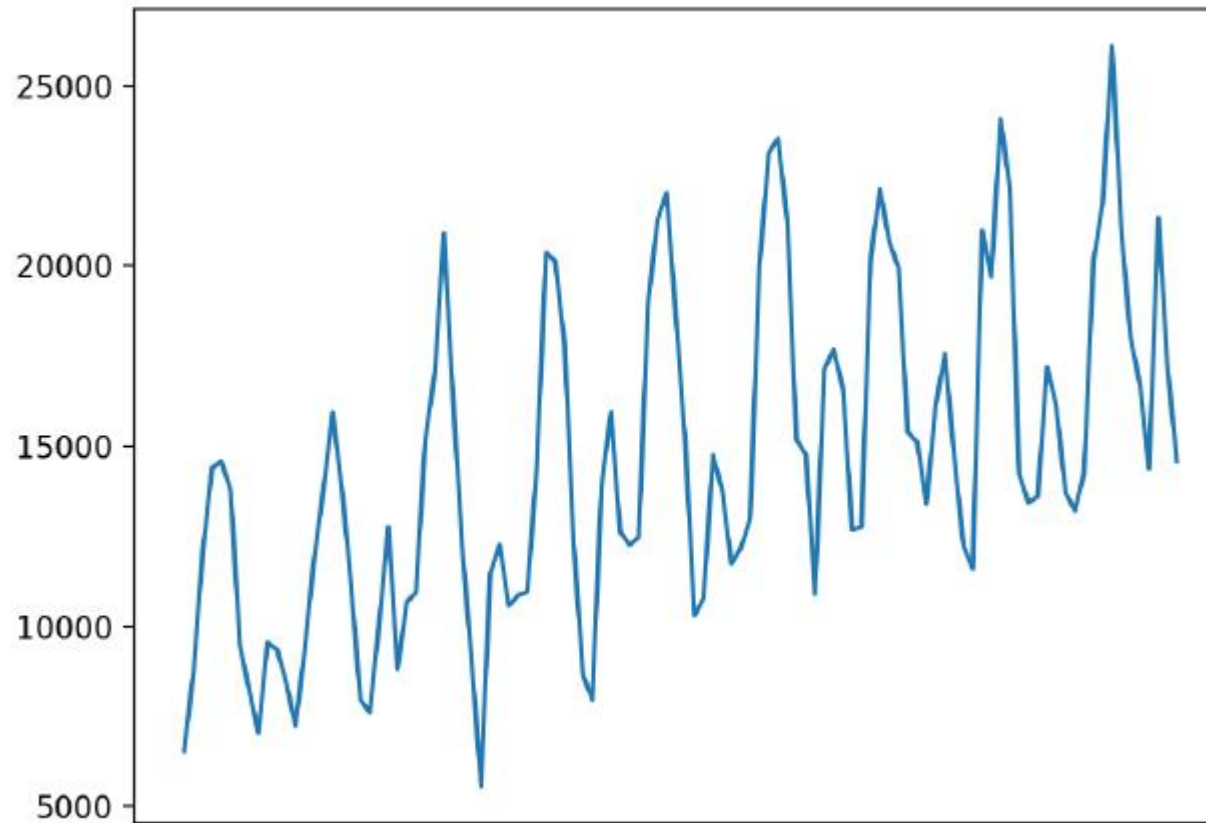
- An up-and-down repetitive movement within a trend occurring periodically
- A *seasonal* pattern occurs when a time series is affected by seasonal factors such as the time of the year or the day of the week
- Seasonality is always of a fixed and known frequency



Components of TSA



Trend and seasonal components



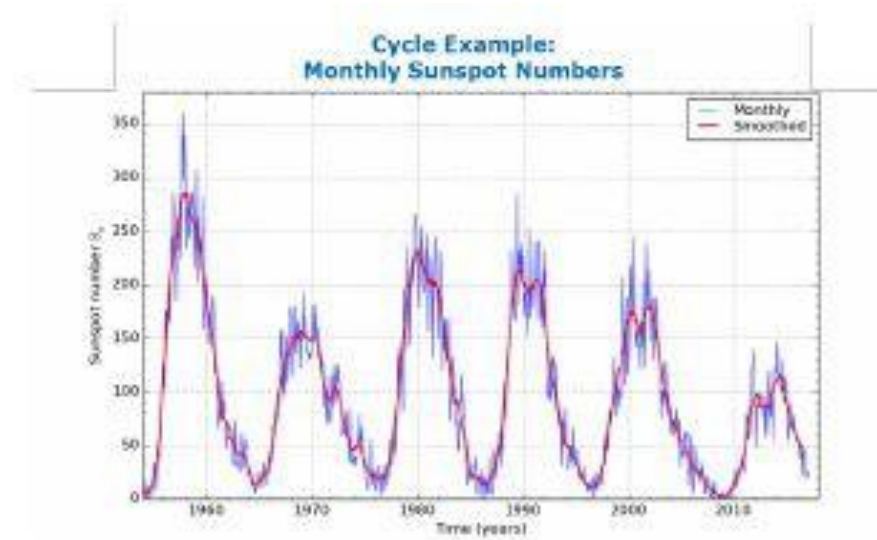
Line Plot of the Monthly Car Sales Dataset

Components of TSA



Cycle

- An up-and-down repetitive movement that are not of a fixed frequency
- Repeats itself over a long period of time
- Fluctuations are usually due to economic conditions, and are often related to the “business cycle”.



Cyclic Vs Seasonal

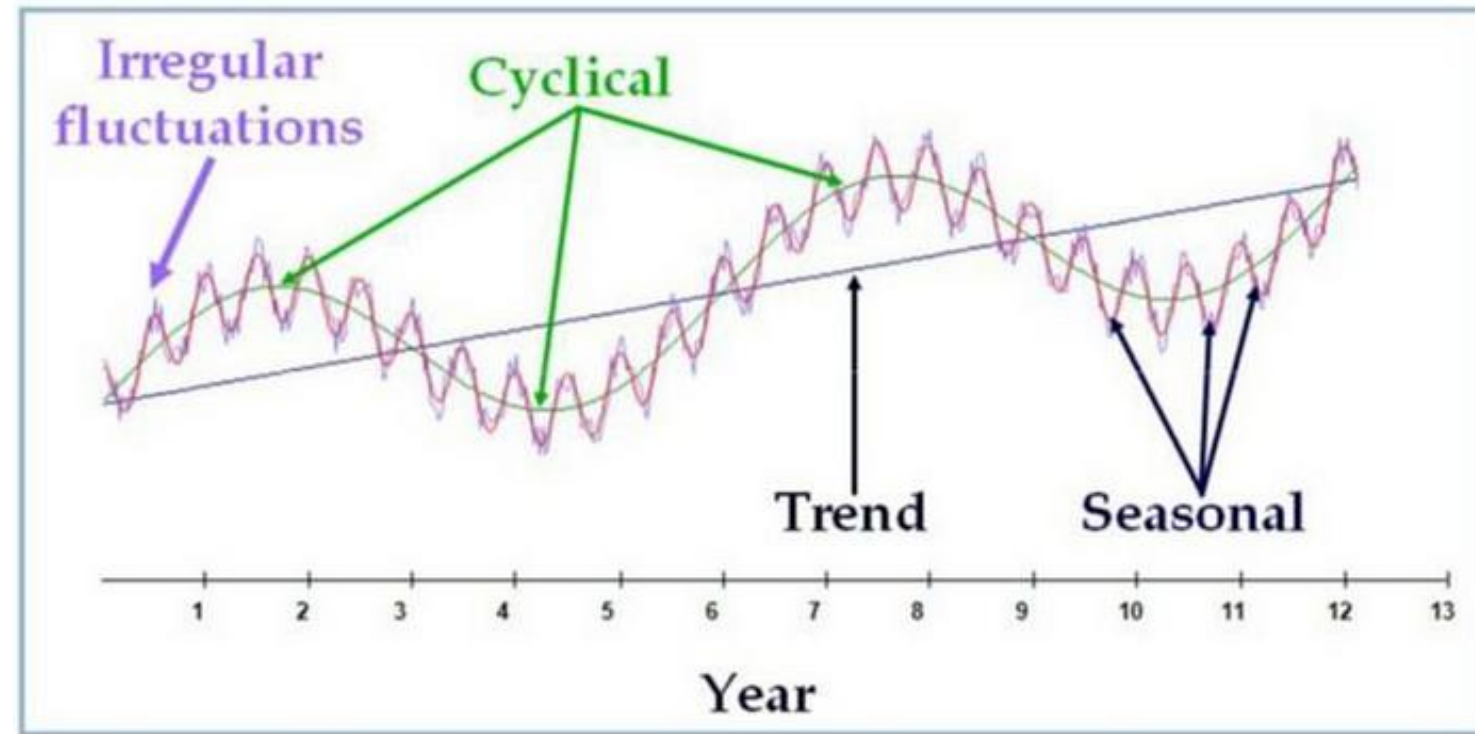
- If the fluctuations are not of a fixed frequency then they are cyclic
- if the frequency is unchanging and associated with some aspect of the calendar, then the pattern is seasonal.
- In general, the average length of cycles is longer than the length of a seasonal pattern
- Magnitudes of cycles tend to be more variable than the magnitudes of seasonal patterns

Components of TSA



- RESIDUAL
 - The residual/error component is everything that is not considered in previous components.
 - Typically, it is assumed to be the sum of a set of random factors (e.g. a white noise series) not relevant for describing the dynamics of the series.

When choosing a forecasting method, we will first need to identify the time series patterns in the data, and then choose a method that is able to capture the patterns properly.



Time series decomposition



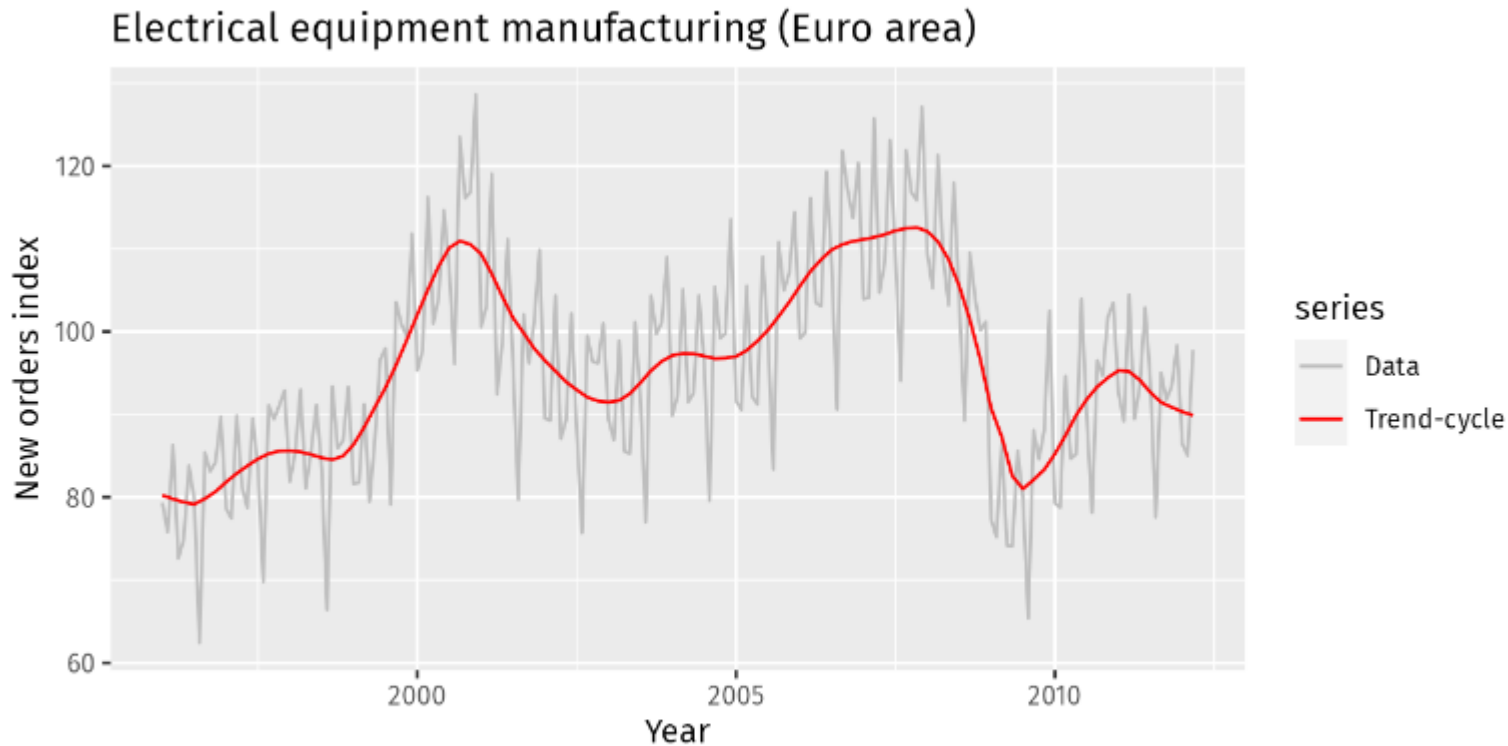
- The **decomposition of time series** is a statistical task that deconstructs a time series into several components, each representing one of the underlying categories of patterns
- The constituent components are trend, seasonality, and residual
- Trend and cycle is usually combined into a single **trend-cycle** component
- Each of the decomposed components or any combination of decomposed components are applied for building a model

Time series decomposition



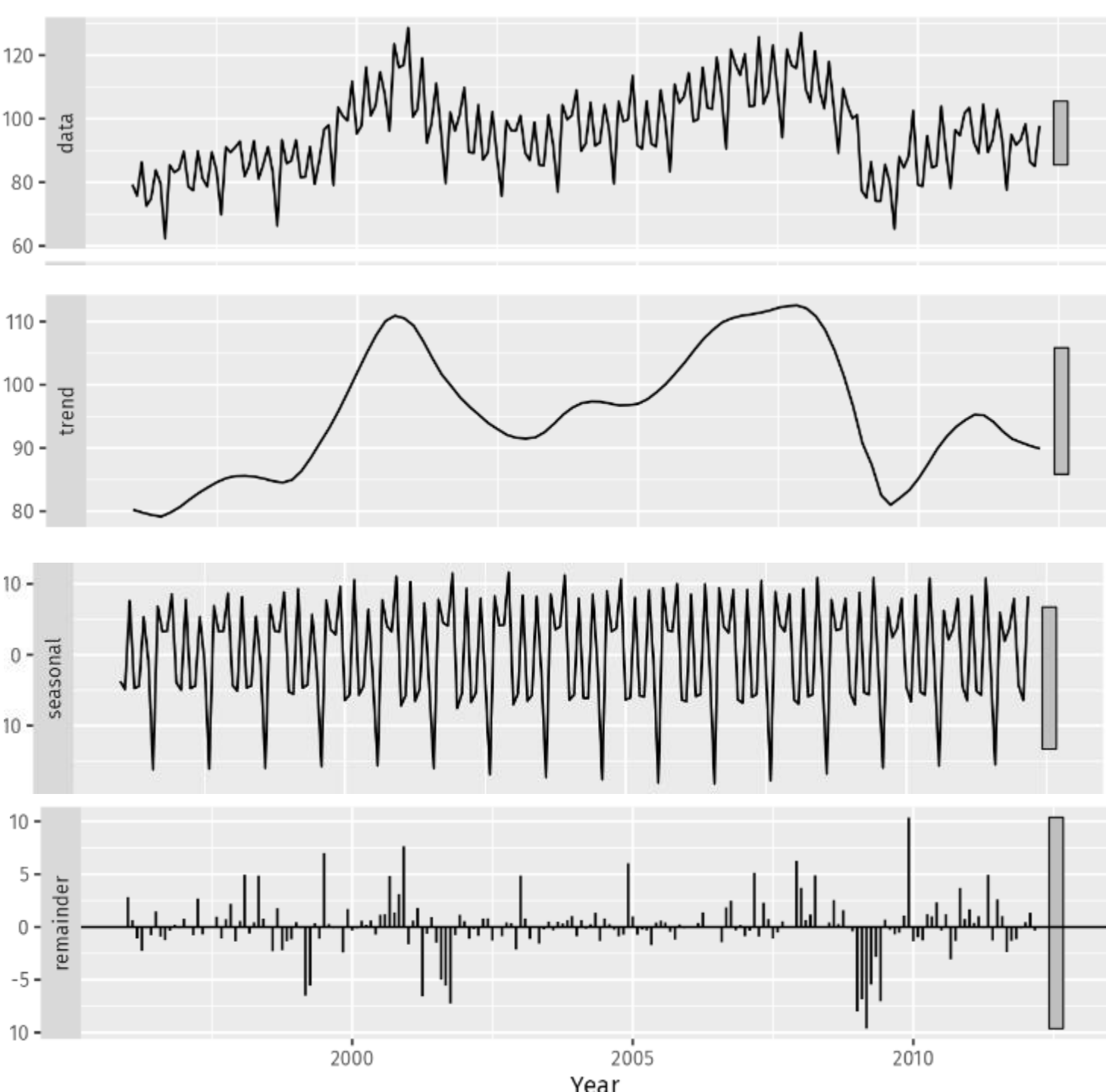
- Let y_t = data, S_t = seasonal component, T_t = trend-cycle component, R_t = remainder component all at period t
- Additive decomposition
 - $y_t = S_t + T_t + R_t$
 - Most appropriate if the magnitude of the seasonal fluctuations, or the variation around the trend-cycle, does not vary with the level of the time series
- Multiplicative decomposition
 - $y_t = S_t \times T_t \times R_t$
 - Most appropriate if the magnitude variation around the trend-cycle, appears to be proportional to the level of the time series
 - common with economic time series

Case study

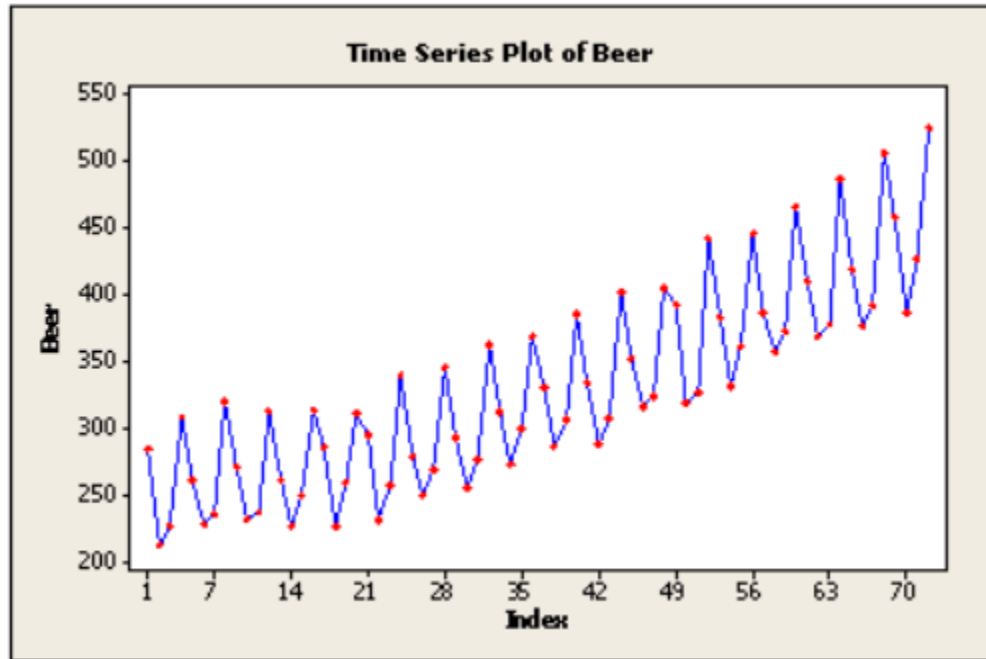


The data show the number of new orders for electrical equipment (computer, electronic and optical products) in the Euro area (16 countries). The data have been adjusted by working days and normalised so that a value of 100 corresponds to 2005.

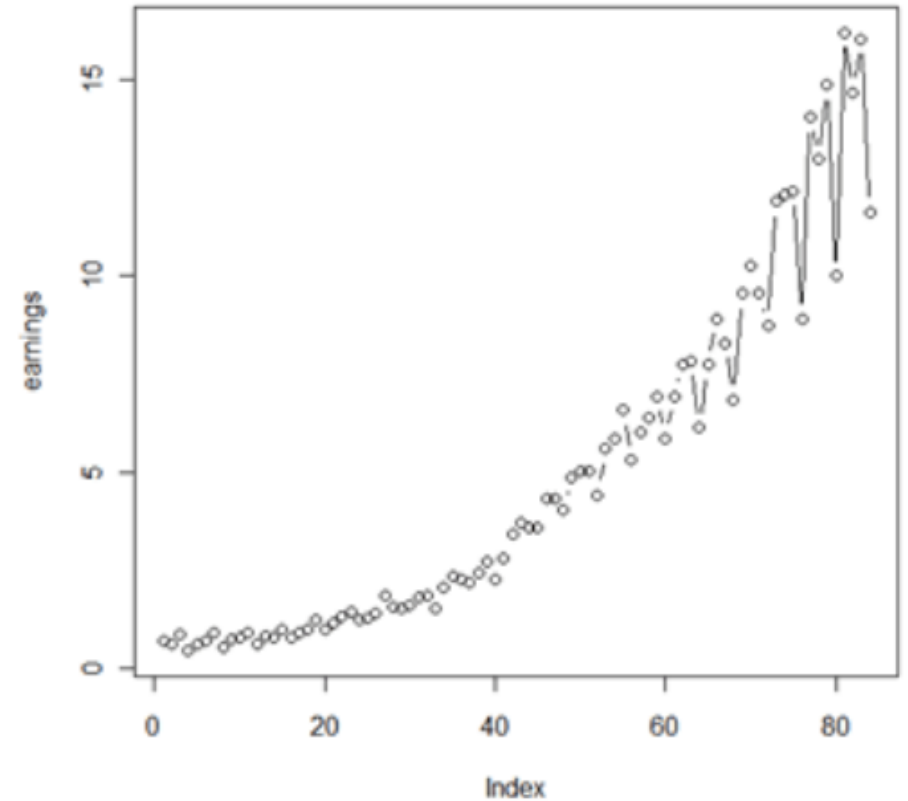
Figure 6.1: Electrical equipment orders: the trend-cycle component (red) and the raw data (grey).



Time series decomposition



Additive model is useful when the seasonal variation is relatively constant over time



Multiplicative model is useful when the seasonal variation increases over time.

Deep Learning for time series forecasting



- Technically, the available context of the sequence provided as input may allow neural network models to learn both trend and seasonality directly
- Neural networks learn arbitrary mapping functions
- **Robust to Noise.** Neural networks are robust to noise in input data can support learning and prediction in the presence of missing values
- **Nonlinear.** Neural networks do not make strong assumptions about the mapping function and readily learn linear and nonlinear relationships
- Neural networks support multivariate inputs.
- Neural networks support multi-step outputs
- Convolutional neural networks support efficient feature learning
- LSTM networks support efficient learning of temporal dependencies
- Hybrid models efficiently combine the diverse capabilities of different architectures

Prepare Time Series Data for DNNs



- Transform a time series dataset into a two-dimensional supervised learning format.
- Transform a two-dimensional time series dataset into a three-dimensional structure suitable for CNNs and LSTMs
- Using sliding window method
- Split a very long time series into subsequences ready for training a CNN or LSTM model

Time Series Data to 2D Supervised Learning Data



Univariate Time Series Data

time,	measure
1,	100
2,	110
3,	108
4,	115
5,	120

Univariate Time Series Data for supervised learning

X,	y
?,	100
100,	110
110,	108
108,	115
115,	120
120,	?

Time Series Data to 2D Supervised Learning Data



Multivariate Time Series Data

time,	measure1,	measure2
1,	0.2,	88
2,	0.5,	89
3,	0.7,	87
4,	0.4,	88
5,	1.0,	90

Multivariate Time Series Data for supervised learning

X1,	X2,	X3,	y
?,	?,	0.2,	88
0.2,	88,	0.5,	89
0.5,	89,	0.7,	87
0.7,	87,	0.4,	88
0.4,	88,	1.0,	90
1.0,	90,	?,	?

Time Series Data to 2D Supervised Learning Data



Multivariate Time Series Data

time,	measure1,	measure2
1,	0.2,	88
2,	0.5,	89
3,	0.7,	87
4,	0.4,	88
5,	1.0,	90

Multivariate Time Series Data for sequence prediction (multi-step)

X1,	X2,	y1,	y2
?,	?,	0.2,	88
0.2,	88,	0.5,	89
0.5,	89,	0.7,	87
0.7,	87,	0.4,	88
0.4,	88,	1.0,	90
1.0,	90,	?,	?

References



- Neural Architecture Search: A Survey
- <https://www.jmlr.org/papers/volume20/18-598/18-598.pdf>
- A Survey on Neural Architecture Search
<https://arxiv.org/pdf/1905.01392>
- Deep Learning for Time Series Forecasting by Jason Brownlee (R4)
- **Forecasting: Principles and Practice (2nd ed)** by Rob J Hyndman and George Athanasopoulos - Monash University, Australia